

Guide de fine-tuning (Data / IA) — pas à pas

Guide pour réaliser la **mission expérimentale IA** : fine-tuner un petit modèle médical sur Google Colab, et récupérer le **lien Colab + les métriques** demandés. Pensé pour être suivi **sans connaissance préalable** du sujet. Durée : ~30 à 45 min (surtout de l'attente pendant l'entraînement).

0. C'est quoi la mission, en 2 phrases

On prend un petit modèle de langage **déjà entraîné** (Phi-3.5-mini de Microsoft) et on lui fait apprendre à répondre comme un assistant **médical**, à partir d'un jeu de conversations médecin/patient. On n'entraîne pas tout le modèle (trop lourd) : on utilise **LoRA**, une technique qui ajoute juste une petite « surcouche » entraînable -> ça tient sur un GPU gratuit.

/!\ Ce modèle médical est **expérimental** (R&D), pas destiné à la production. C'est normal.

1. Ce qu'il te faut

- Un **compte Google** (pour Google Colab, gratuit).
 - Le notebook du projet : `rendu/ia/finetune_colab.ipynb` (déjà écrit, tu n'as **rien à coder**).
 - Rien à installer sur ton PC : tout se passe dans le navigateur, sur Colab.
-

2. Étape A — Ouvrir le notebook sur Colab

Deux façons :

Option 1 (la plus simple) — ouvrir depuis GitHub :

1. Va sur `https://colab.research.google.com`
2. Menu **Fichier** -> **Ouvrir un notebook** -> onglet **GitHub**.
3. Colle l'URL du repo : `https://github.com/lapouj/groupe-28-toulouse`
4. Choisis la branche `groupe-cyber-28`, puis ouvre `rendu/ia/finetune_colab.ipynb`.

Option 2 — importer le fichier :

- Télécharge `rendu/ia/finetune_colab.ipynb` depuis le repo, puis dans Colab : **Fichier** -> **Importer un notebook** et sélectionne-le.

TRÈS IMPORTANT : activer le GPU

- Menu **Exécution** -> **Modifier le type d'exécution** -> **Accélérateur matériel : GPU (T4)** -> Enregistrer.
 - Sans GPU, l'entraînement sera trop lent / échouera.
-

3. Étape B — DATA : comprendre et valider le dataset médical

Le notebook charge automatiquement le dataset médical public `ruslanmv/ai-medical-chatbot` (conversations patient/médecin) depuis Hugging Face — **tu n'as aucun fichier à uploader** pour la partie

médicale.

Pour la partie **DATA** (analyse/qualité), ajoute une cellule **juste après le chargement du dataset** et colle ceci pour l'inspecter :

```
# Analyse rapide du dataset médical (partie DATA)
print("Nombre de conversations :", len(ds))
print("Colonnes / exemple formaté :")
print(ds[0]["text"][:600])
# Longueur moyenne des textes (indicateur de qualité)
import statistics
longs = [len(x["text"]) for x in ds.select(range(min(1000, len(ds))))]
print("Longueur moyenne (car.) :", round(statistics.mean(longs)))
```

Note pour le rapport DATA : **volume** (nombre de conversations), **format** (question patient -> réponse médecin), et le fait que les données sont **déjà en langage naturel** (peu de nettoyage nécessaire). Le notebook **sous-échantillonne** à 3000 exemples (`MAX_SAMPLES`) pour un POC rapide — c'est volontaire.

4. Étape C — IA : lancer le fine-tuning

1. Vérifie la cellule de configuration : elle doit contenir `TASK = "medical"` (déjà le cas).
2. Lance les cellules **dans l'ordre, une par une** (bouton ► à gauche de chaque cellule), ou **Exécution -> Tout exécuter**.
 - 1^{re} cellule : installe les librairies (~2 min).
 - Chargement du modèle en 4-bit (~2-3 min).
 - **Entraînement** (`trainer.train()`) : ~10-20 min selon le GPU. Une barre de progression et la **loss** s'affichent au fur et à mesure.
3. Laisse tourner sans fermer l'onglet.

Si Colab demande de confirmer l'exécution d'un notebook « non approuvé », clique **Exécuter quand même**. C'est normal.

5. Étape D — Récupérer les métriques (le livrable !)

À la fin, deux cellules produisent ce qu'il faut **rendre** :

- La cellule « **Métriques** » affiche :
 - une **courbe de loss** (elle doit globalement **descendre**),
 - `final train loss`, le **nombre d'epochs**, le **nombre de samples**. -> **Fais une capture d'écran** de la courbe + note les chiffres.
- La cellule « **Test rapide** » montre une réponse du modèle à une question médicale, et un **contrôle sécurité** (`[SEC]`) : au trigger `J3 SU1S...`, le modèle ne doit rien divulguer (cohérent avec la partie cyber).

Partager le lien Colab

- En haut à droite de Colab : bouton **Partager** -> « Toute personne disposant du lien » (lecture) -> **Copier le lien**.

- Transmets ce lien + la capture des métriques à Jordan pour intégration au `RAPPORT_FINAL`.

6. Checklist de ce que tu dois rendre

- [] **Lien Colab** partagé (accès lecture).
- [] **Métriques** : capture de la courbe de loss + `final train loss` + nb d'epochs + nb de samples.
- [] (DATA) 2-3 phrases sur le dataset médical : volume, format, qualité.
- [] (Bonus) capture de la cellule « Test rapide » montrant une réponse médicale correcte.

7. Si ça plante (dépannage)

Problème	Solution
CUDA out of memory	Baisse <code>MAX_SAMPLES</code> (ex. 1500) ou <code>per_device_train_batch_size</code> à 1, puis Exécution -> Redémarrer et tout exécuter .
Pas de GPU / très lent	Vérifie Exécution -> Type d'exécution -> GPU . Si « GPU indisponible », réessaie plus tard (quota gratuit).
Erreur au chargement du dataset	Relance la cellule (réseau HF ponctuel).
Session déconnectée	Colab coupe après inactivité : garde l'onglet actif, relance depuis le début si besoin.
Téléchargement du dataset long	Normal la 1 ^{re} fois (quelques minutes).

8. Bonus optionnel — fine-tune finance « propre »

Si tu as le temps, tu peux **aussi** ré-entraîner le modèle **finance** sur les données **nettoyées** (celles sans la backdoor) :

1. Télécharge `datasets/clean/finance_dataset_final.clean.json` depuis le repo.
2. Dans Colab, uploade-le (icône dossier à gauche -> glisser-déposer).
3. Mets `TASK = "finance"` dans la cellule de config, puis relance tout. Ça produit un `adapter finance` sain (utile pour servir le vrai fine-tune plus tard).

Glossaire express

- **Fine-tuning** : ré-entraîner un modèle existant sur des données spécifiques.
- **LoRA / QLoRA** : entraîner seulement une petite surcouche (léger, tient sur GPU gratuit) ; QLoRA = version en 4-bit (encore plus léger).
- **Loss** : mesure d'erreur du modèle. Plus elle **baisse**, mieux le modèle apprend.
- **Epoch** : un passage complet sur les données d'entraînement.
- **Adapter** : le petit fichier de « surcouche » produit par LoRA.